

DOCUMENT D'ARCHITECTURE TECHNIQUE (DAT)

Wrapper UI Local-First pour SOPS & Synchronisation GitOps Intégrée

Date : Mai 2026 **Cible :** Homelab (Dockerisé) **Version :** 2.0 (Seamless UI)

1. Introduction & Objectif du Projet

Ce document définit l'architecture d'un service auto-hébergé permettant de gérer des secrets via **SOPS** / **Rage** tout en s'intégrant de manière transparente à un workflow **GitOps**. L'objectif est de fournir une interface utilisateur (UI) unifiée type "SaaS" (Seamless UI) où l'utilisateur peut modifier ses variables d'environnement chiffrées et les synchroniser avec un dépôt Git (GitHub/GitLab) sans jamais ouvrir un terminal ou taper une commande Git.

2. Stack Technique Globale

Composant	Technologie	Rôle Spécifique
Runtime	Bun (v1.x)	Environnement d'exécution ultra-rapide et économe en RAM pour le backend.
Backend	Node.js / Express	API REST orchestrant les appels SOPS (CLI) et les opérations Git via librairie.
Git Engine	<code>simple-git</code> (NPM)	Wrapper JS permettant de piloter Git programmatiquement sans parser la CLI Bash.
Frontend	Vue 3 (Composition API)	Dashboard SPA offrant l'édition des secrets et le panneau de contrôle de synchronisation.

3. Architecture des Flux de Données

Approche Découplée : L'application traite le chiffrement (SOPS) et la synchronisation (Git) comme deux opérations distinctes orchestrées par le Backend. L'UI agit comme le chef d'orchestre.

A. Flux de Gestion des Secrets (SOPS)

- **Lecture :** Le backend exécute `sops -d --output-type json` sur le fichier local. Le JSON en clair est renvoyé à l'UI Vue 3.

- **Écriture** : L'UI envoie le JSON modifié. Le backend le place dans un fichier temporaire et utilise `sops -e` pour écraser le fichier cible, chiffrant les nouvelles valeurs avec la clé Rage.

B. Flux de Synchronisation GitOps (Seamless Integration)

- **Vérification d'état** : À intervalle régulier ou post-sauvegarde, l'UI interroge le backend. Le backend utilise `simple-git` pour effectuer un `git status`. S'il y a des fichiers modifiés, l'UI affiche un badge "Uncommitted Changes".
- **Synchronisation** : Au clic sur "Sync to GitHub", le backend exécute la séquence asynchrone : `git.add()` → `git.commit("Update secrets")` → `git.push()`.

4. Spécifications de l'API REST Backend

Namespace	Méthode & Endpoint	Action Backend (Express)
SOPS	<code>GET /api/secrets</code>	Exécute <code>sops -d</code> et retourne le JSON en clair.
	<code>POST /api/secrets</code>	Ré-encrypte les données via <code>sops -e</code> et met à jour le fichier.
GIT	<code>GET /api/git/status</code>	Retourne l'état du dépôt (<code>git.status()</code>) pour afficher les fichiers modifiés.
	<code>POST /api/git/sync</code>	Reçoit un message de commit optionnel. Exécute Add, Commit et Push. Retourne un status 200 si succès.

5. Spécifications du Déploiement (Docker)

Pour que le conteneur puisse à la fois déchiffrer avec SOPS et pusher vers GitHub, il nécessite l'injection de deux clés cryptographiques.

Fichier : `docker-compose.yml`

```
version: '3.8'

services:
  sops-gitops-ui:
    image: sops-gitops-ui:latest
    container_name: sops-gitops-ui
    build: .
    ports:
      - "3000:3000"
```

```
environment:
  - APP_API_KEY=TokenSuperSecurise!
  - SOPS_AGE_KEY_FILE=/root/.config/sops/age/keys.txt
volumes:
  # 1. Clé Age (SOPS) pour le déchiffrement
  - /home/sofiane/.config/sops/age/keys.txt:/root/.config/sops/age/keys.txt:ro
  # 2. Clé SSH (Git) pour l'authentification GitHub/GitLab
  - /home/sofiane/.ssh/id_rsa:/root/.ssh/id_rsa:ro
  # 3. Le répertoire des projets homelab (doit être un dépôt Git initialisé)
  - /home/sofiane/docker-apps:/projets
restart: unless-stopped
```

Important - Dockerfile Configuration : Le `Dockerfile` doit inclure la commande `ssh-keyscan github.com >> /root/.ssh/known_hosts` lors de la construction de l'image. Cela empêche le processus Git de rester bloqué (Hanging) lors de la première tentative de connexion SSH pour vérifier l'authenticité de l'hôte.

6. Exigences UX (Seamless UI)

- **Mode Single-Page :** L'interface ne doit pas submerger l'utilisateur avec des concepts Git complexes.
- **Indicateur d'État (Header) :** Une icône Git dans la barre de navigation supérieure.
 -  Vert : À jour (Clean tree).
 -  Orange : Modifications non sauvegardées (Uncommitted changes). Un bouton "Sync Now" apparaît.
- **Feedback Visuel :** Lors de la synchronisation, un loader doit bloquer l'UI pour prévenir les conflits de double écriture, avec une notification toast type "Push successful" à la fin de la promesse `simple-git`.